BINF 4070 / COMS 4995 W008

The Future of Personal Health Assistant

Fall 2026

Data Models for Health Data

Week 05

Dr. Xuhai “Orson” Xu | Dr. Eugene Wu

AI @ VP&S Initiative

COLUMBIA | VAGELOS COLLEGE OF PHYSICIANS AND SURGEONS

COLLABORATORY AT COLUMBIA UNIVERSITY

DAP Lab

Today's Outline

1. **Data Models**
2. **Relational Data Management**
3. **Document Stores**
4. **Information Extraction, Integration, and Cleaning**

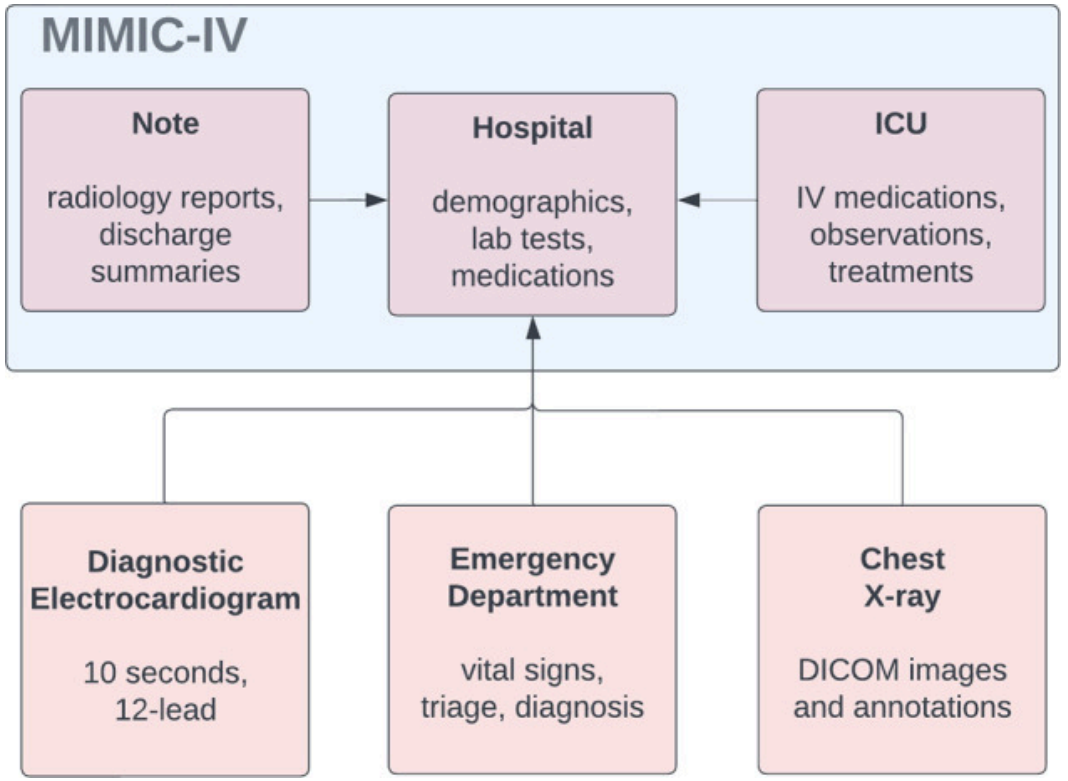
Data Models

Data sources, data models, syntax, and semantics.

MIMIC-IV Modules and Data Types

- **Hospital and ICU:** linked tables of admissions, lab tests, and medications
- **Note:** free-text discharge summaries and radiology reports
- **ECG:** 10-second, 12-lead waveforms
- **Chest X-ray:** DICOM images and annotations

Each module has different queries, sizes, and update patterns.



MIMIC-IV modules — Johnson et al., Scientific Data (2023)

All of Us: Data Types and Storage

Data type	Stored as	One row is
EHR	OMOP tables, e.g. `measurement`	one coded event
Surveys	OMOP `observation`	one answer
Fitbit	custom tables, e.g. `steps_intraday`	one person-minute
Clinical notes	OMOP `note`, `note_nlp`	one note; one extracted concept
Genomes	VCF, Hail, BGEN, PLINK files	not rows: variant files

Release 9 (2026): 747,000+ participants; EHR for 481,000, Fitbit for 68,000, genomes for 535,000.

Example: One Person's Health Data Sources

The question: "Has my HbA1c improved since I started metformin?"

Source	Data model	Lecture
EHR labs	relational	today

```
lab_results
lab_id | loinc  | value | unit | collected
L044   | 4548-4 | 8.2   | %    | 2026-03-15
L051   | 4548-4 | 7.6   | %    | 2026-06-20
```

Rows with the same columns.

Example: One Person's Health Data Sources

The question: "Has my HbA1c improved since I started metformin?"

Source	Data model	Lecture
EHR labs	relational	today
Portal record	document	today

```
{
  "resourceType": "Condition",
  "subject": {"reference": "Patient/P001"},
  "code": {"coding": [{
    "system": "http://snomed.info/sct",
    "code": "44054006",
    "display": "Type 2 diabetes mellitus"
  }]},
  "onsetDateTime": "2025-08-20"
}
```

One nested object per resource.

Example: One Person's Health Data Sources

The question: "Has my HbA1c improved since I started metformin?"

Source	Data model	Lecture
EHR labs	relational	today
Portal record	document	today
Watch heart rate	time series	Week 7

```
time                 heart rate (/min)
2026-06-20 08:00:00  72
2026-06-20 08:01:00  75
2026-06-20 08:02:00  118
2026-06-20 08:03:00  121
```

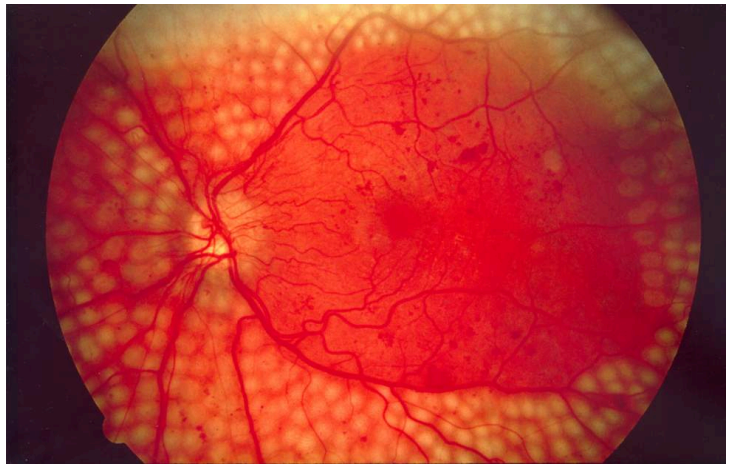
One value per timestamp.

Example: One Person's Health Data Sources

The question: "Has my HbA1c improved since I started metformin?"

Source	Data model	Lecture
EHR labs	relational	today
Portal record	document	today
Watch heart rate	time series	Week 7
Retinal photo	image	Week 7

```
DICOM header: Modality OP, Laterality R
Pixel data:   975 × 621 × 3 values
```



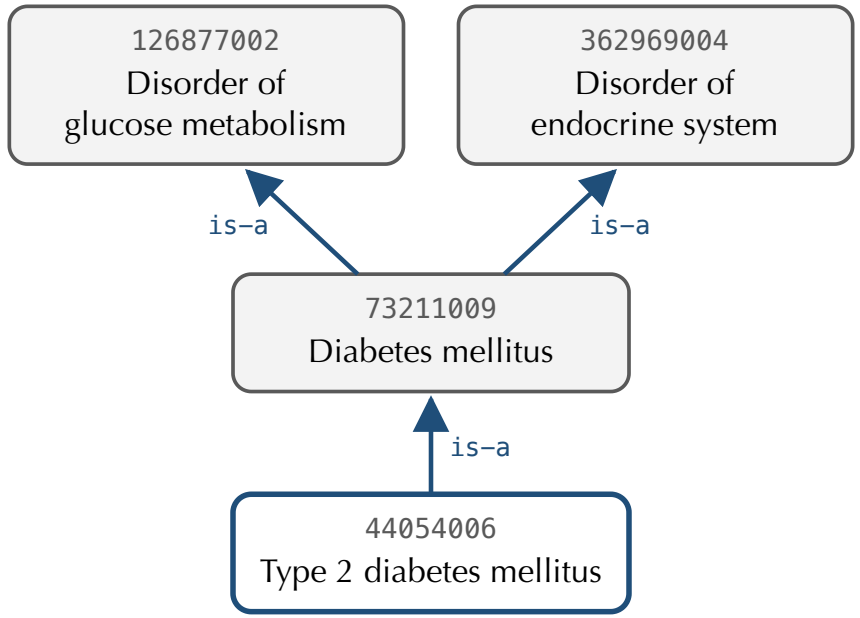
A pixel array plus metadata.

Fundus photo showing scatter laser surgery for diabetic retinopathy — National Eye Institute, NIH

Example: One Person's Health Data Sources

The question: "Has my HbA1c improved since I started metformin?"

Source	Data model	Lecture
EHR labs	relational	today
Portal record	document	today
Watch heart rate	time series	Week 7
Retinal photo	image	Week 7
Diagnosis codes	knowledge graph	later



Concepts linked by typed edges.

Example: One Person's Health Data Sources

The question: "Has my HbA1c improved since I started metformin?"

Source	Data model	Lecture
EHR labs	relational	today
Portal record	document	today
Watch heart rate	time series	Week 7
Retinal photo	image	Week 7
Diagnosis codes	knowledge graph	later
Visit note	text + vectors	Week 6

```
"Started metformin 500 mg twice daily.
 Recheck HbA1c in 3 months."

embedding → [0.021, −0.113, 0.087, …]
             384 numbers
```

Free text, searched by meaning.

Definition: Data Model

A **data model** is a collection of concepts for describing data: its structures, relationships, and constraints, plus the operations that query and update it.

Element of the relational model	Example in `lab_results`
Structure	a table with columns `lab_id`, `patient_id`, `loinc_code`, `value`
Relationship	`patient_id` refers to a row in `patients`
Constraint	`lab_id` is unique; `value` is not null
Operations	`SELECT`, `JOIN`, `INSERT`, `UPDATE`

The DBMS implements the model and manages storage on disk.

Data Models Enforce Syntax

Two HbA1c (hemoglobin A1c) lab reports, mapped to FHIR. Are they comparable?

Report A · US lab

```
HEMOGLOBIN A1C
8.2 %  H  (4.0–5.6)
Collected: 03/15/2026
```

→

FHIR field	A	B
code	4548-4	59261-8
value	8.2	58
unit	%	mmol/mol
date	2026-03-15	2026-06-05

←

Report B · clinic abroad

```
HbA1c 58 mmol/mol
(IFCC). Sample
taken 05/06/2026.
```

FHIR gives both reports the same structure, but their codes and units differ: 8.2 and 58 cannot be compared directly.

Data Models Do Not Enforce Semantics

Report B mapped so that it is syntactically comparable to A. Are they comparable now?

Report A · US lab

```
HEMOGLOBIN A1C
8.2 %  H  (4.0–5.6)
Collected: 03/15/2026
```

→

FHIR field	A	B
code	4548-4	4548-4 ✗
value	8.2	58
unit	%	% ✗
date	2026-03-15	2026-05-06 ✗

←

Report B · clinic abroad

```
HbA1c 58 mmol/mol
(IFCC). Sample
taken 05/06/2026.
```

Every field has a valid type and code, so the resource passes FHIR validation. It claims an HbA1c of 58% in May; the test was 7.5% in June. A data model fixes the syntax; the meaning depends on codes, units, and conventions.

The Rest of the Lecture

Part	Data systems	For the HbA1c question
Relational data systems	tables, SQL, and transactions, and SQL that now calls AI models	store results and medications; query and check them
Document data systems	nested records such as FHIR resources	keep portal records whole, as received
Data integration	extraction, normalization, and cleaning across sources	turn the two reports into comparable rows

Relational Data Management

The relational model, SQL, transactions, and semantic operators.

Data Models and File Formats

A **file format** specifies how bytes are laid out. A **data model** specifies what structures, relationships, and operations the data support.

Data model	Possible file formats
Relational table	CSV, Parquet, a SQLite database file
FHIR resource	JSON, XML, RDF Turtle
Apple Health records	one XML file whose `<Record>` elements are flat rows

The file format does not determine the entities, keys, or constraints.

Quiz: A Crashed Import

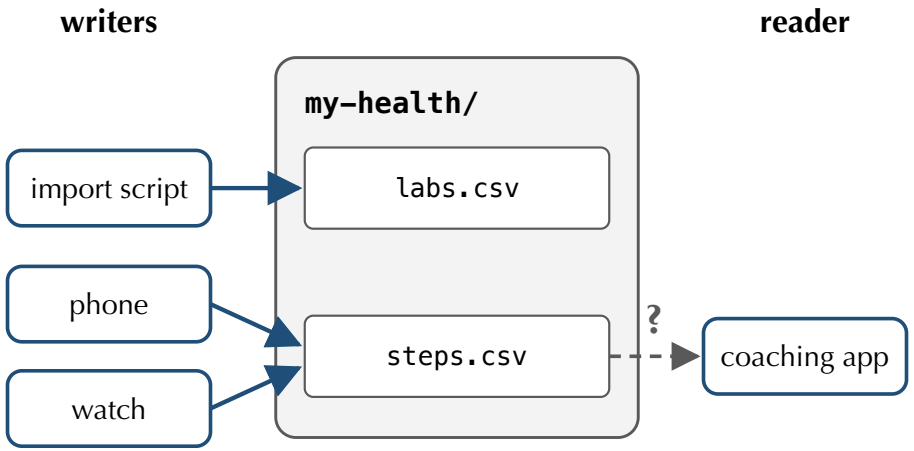
`labs.csv` starts empty. The import script appends 100 lab rows and crashes after row 40. You rerun it, and it finishes. How many rows does `labs.csv` hold?

A. 40

B. 100

C. 140

D. 🤷



Quiz: A Crashed Import

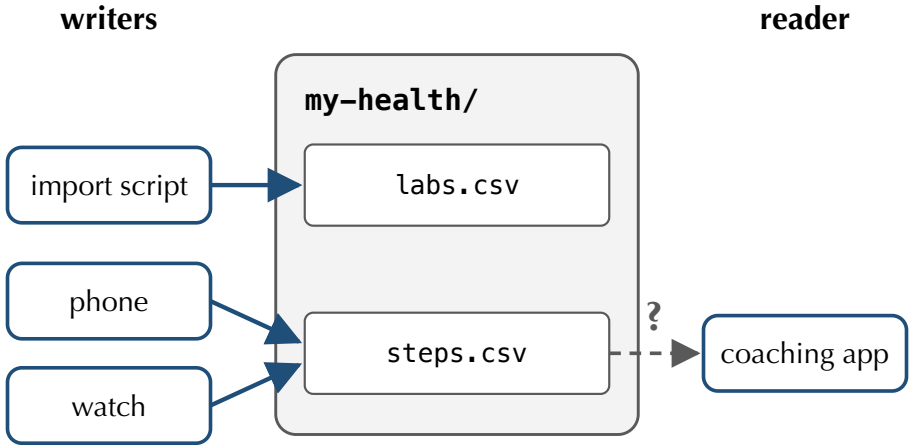
`labs.csv` starts empty. The import script appends 100 lab rows and crashes after row 40. You rerun it, and it finishes. How many rows does `labs.csv` hold?

A. 40

B. 100

C. 140

D. 🤷



Limitation: no atomicity. A crash leaves part of the import, and the file does not record how much.

DBMS mechanism: a transaction. A crash before `COMMIT` rolls back every row, so a rerun stores exactly 100.

Quiz: Two Devices Writing at Once

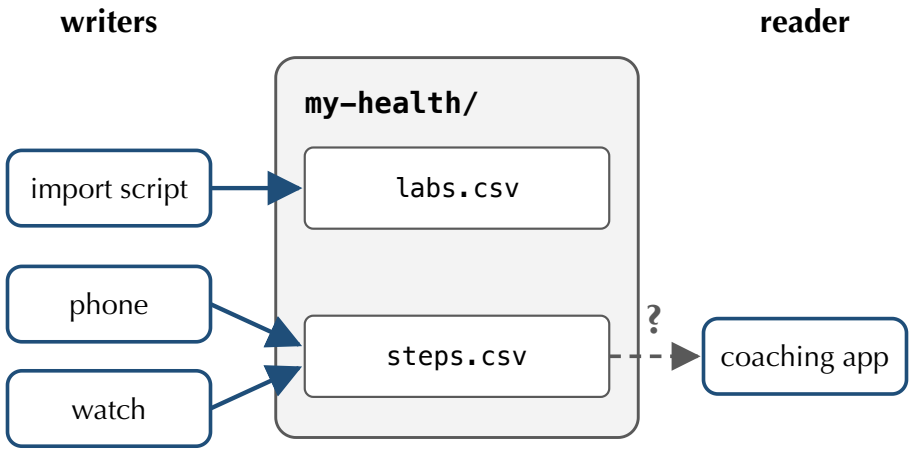
The phone and the watch each append one line to `steps.csv` at the same moment. What can the file end up with?

A. both lines, in either order

B. only one of the lines

C. one line mixing parts of both

D. any of these



Quiz: Two Devices Writing at Once

The phone and the watch each append one line to `steps.csv` at the same moment. What can the file end up with?

A. both lines, in either order

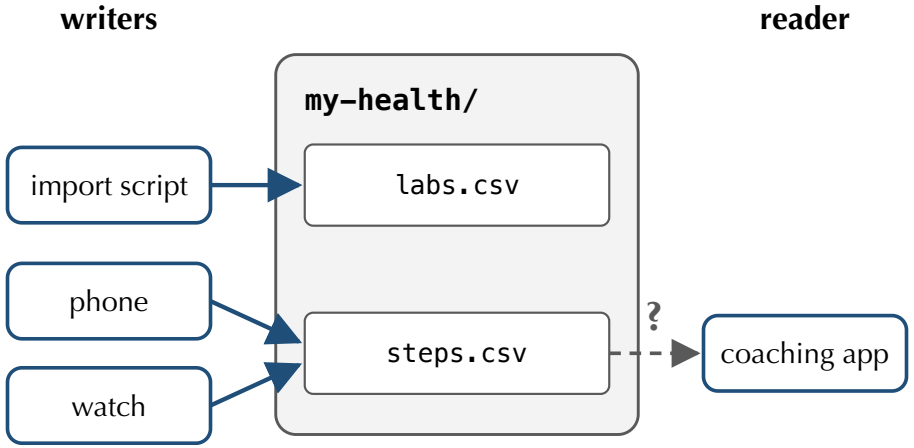
B. only one of the lines

C. one line mixing parts of both

D. **any of these**

Limitation: no concurrency control. The result depends on timing, buffering, and sync software.

DBMS mechanism: isolation. Locking or multiversioning makes concurrent writes behave as if they ran one at a time.



Quiz: A Typo in an HbA1c Value

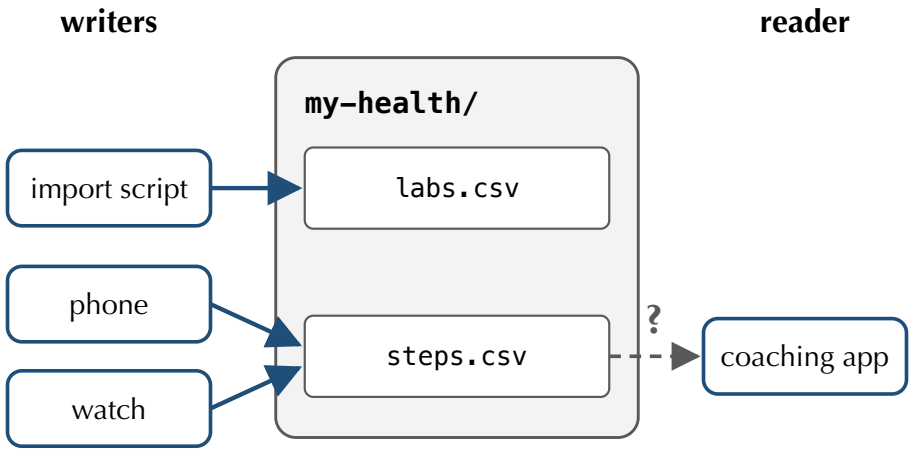
The import script writes an HbA1c of 82 instead of 8.2 to `labs.csv`. What happens?

A. the write fails

B. the file flags the row

C. 82 is saved

D. 82 is rounded to 8.2



Quiz: A Typo in an HbA1c Value

The import script writes an HbA1c of 82 instead of 8.2 to `labs.csv`. What happens?

A. the write fails

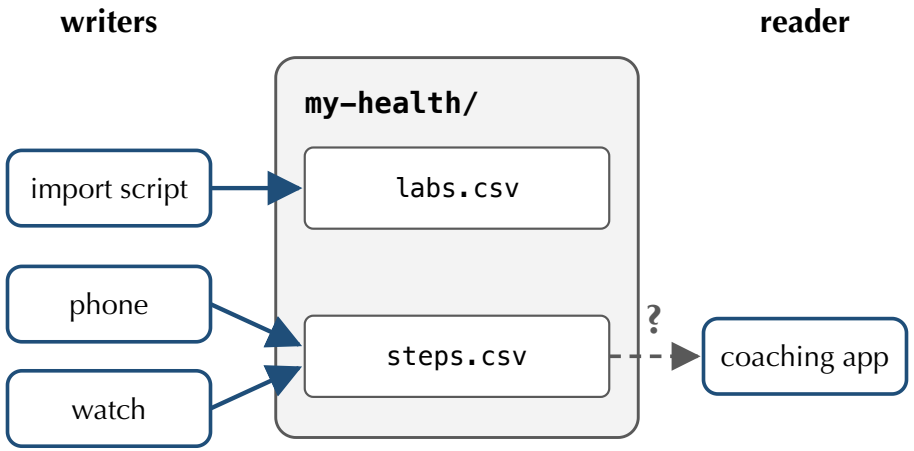
B. the file flags the row

C. **82 is saved**

D. 82 is rounded to 8.2

Limitation: no integrity constraints. A CSV field accepts any text.

DBMS mechanism: consistency. A range `CHECK` on the HbA1c column rejects 82 at insert.



Quiz: Sharing Daily Step Totals

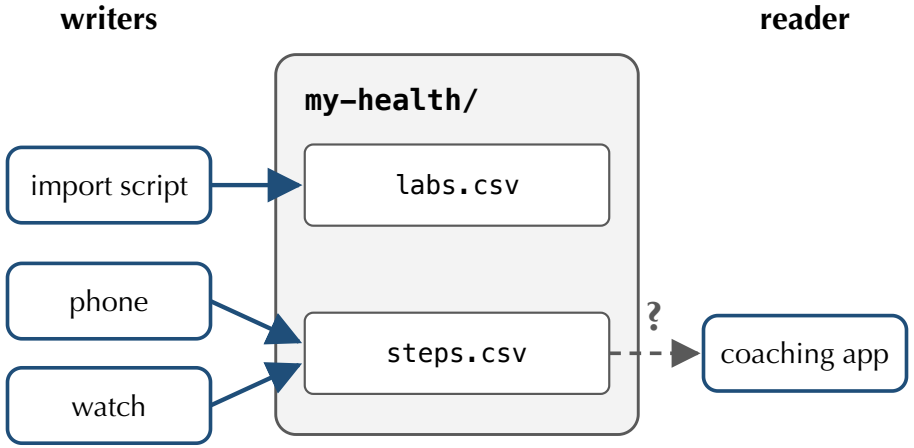
The coaching app should see daily step totals, not minute-by-minute steps. With file permissions, what can you share?

A. daily totals only

B. all of `steps.csv`, or a copy you maintain

C. steps from the last 30 days only

D. nothing



Quiz: Sharing Daily Step Totals

The coaching app should see daily step totals, not minute-by-minute steps. With file permissions, what can you share?

A. daily totals only

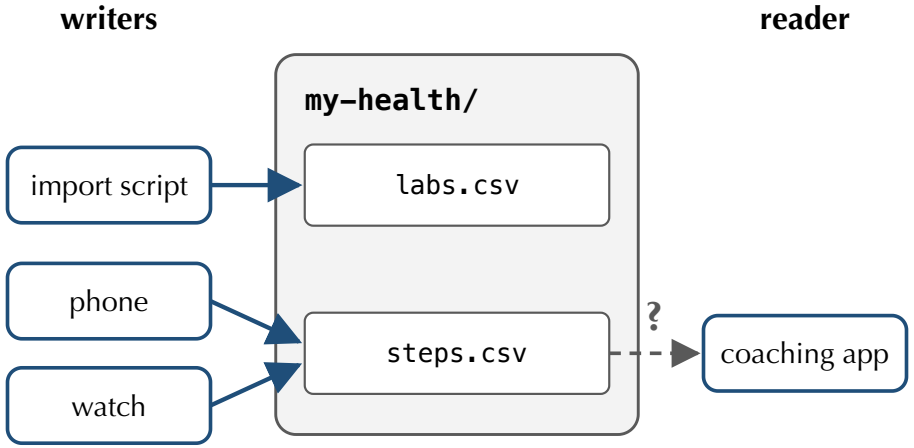
B. **all of `steps.csv`, or a copy you maintain**

C. steps from the last 30 days only

D. nothing

Limitation: coarse access control. Permissions apply to whole files and folders.

DBMS mechanism: access control. `GRANT SELECT` on a view that returns daily totals.



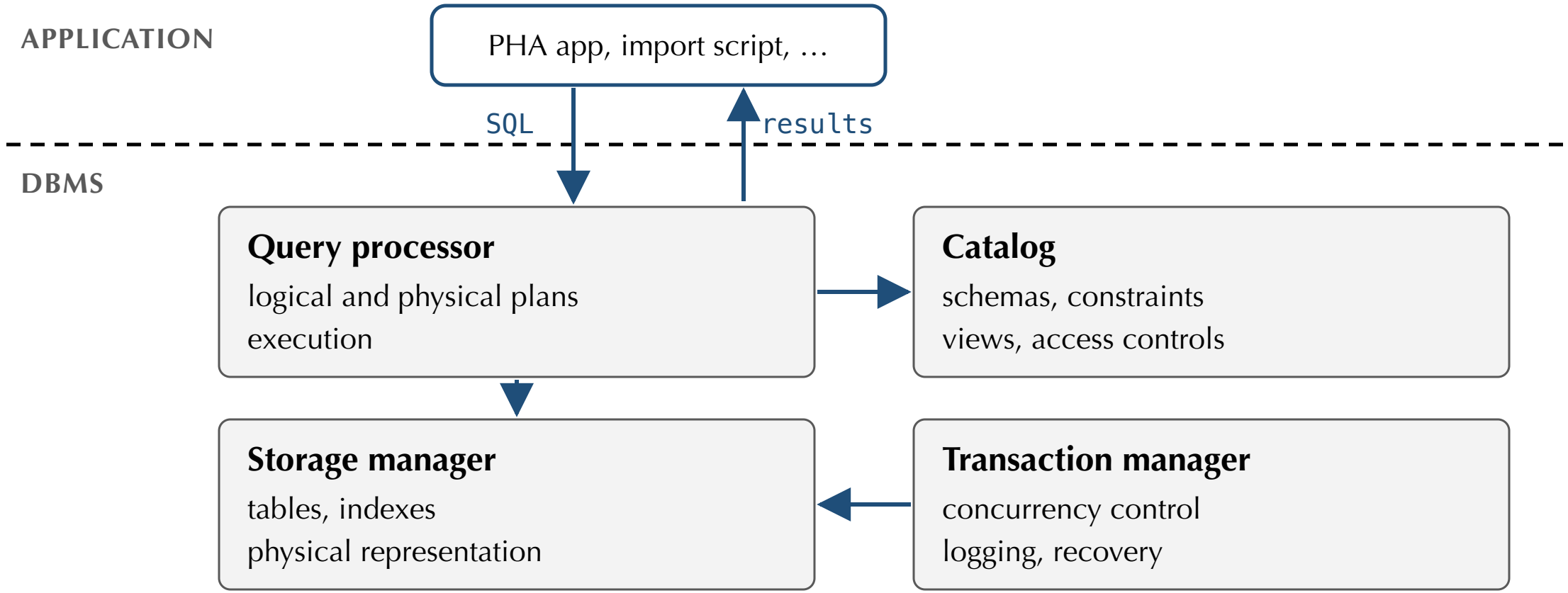
DBMS Mechanisms

Each mechanism, with the example it fixes:

- **Transactions, atomicity:** the crashed import leaves zero rows; a rerun stores exactly 100
- **Transactions, isolation:** the phone's and watch's writes cannot interleave or overwrite each other
- **Consistency:** a `CHECK` on the HbA1c column rejects 82
- **Access control:** the coaching app gets `SELECT` on a view of daily step totals

A **database management system (DBMS)** provides these mechanisms.

Components of a Relational DBMS



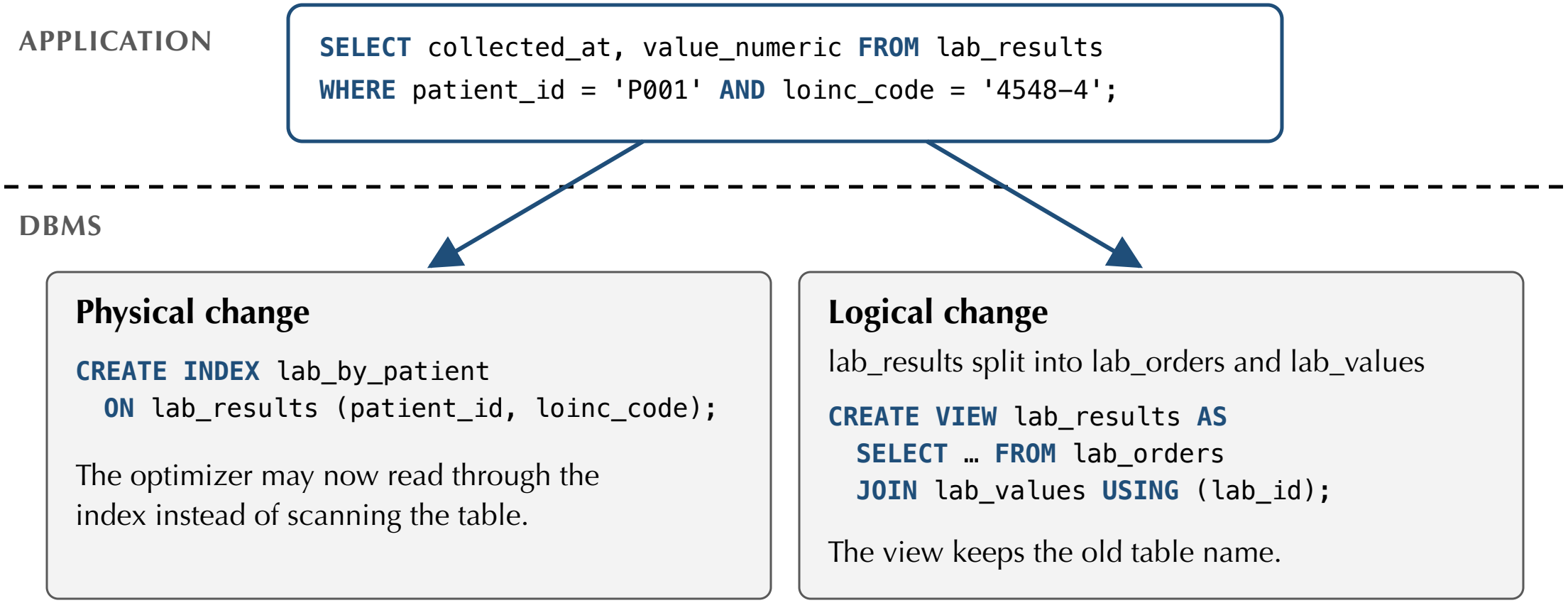
Applications send SQL; the DBMS stores data, enforces rules, and coordinates writers.

Data Independence

> *"Future users of large data banks must be protected from having to know how the data is organized in the machine."*
>
> — E. F. Codd, *A Relational Model of Data for Large Shared Data Banks* (1970)

- **Physical data independence:** add an index or move files without rewriting queries
- **Logical data independence:** a view preserves an app's interface when tables change

Data Independence: Example



The application query runs unchanged after either change below the line.

Declarative Queries

Plan 1: scan every row

```
for each r in lab_results:
  if r.loinc = '4548-4' and r.value > 7:
    output r.patient_id, r.value
```

A program states **how**: read every row, test it, and output the matches.

Declarative Queries

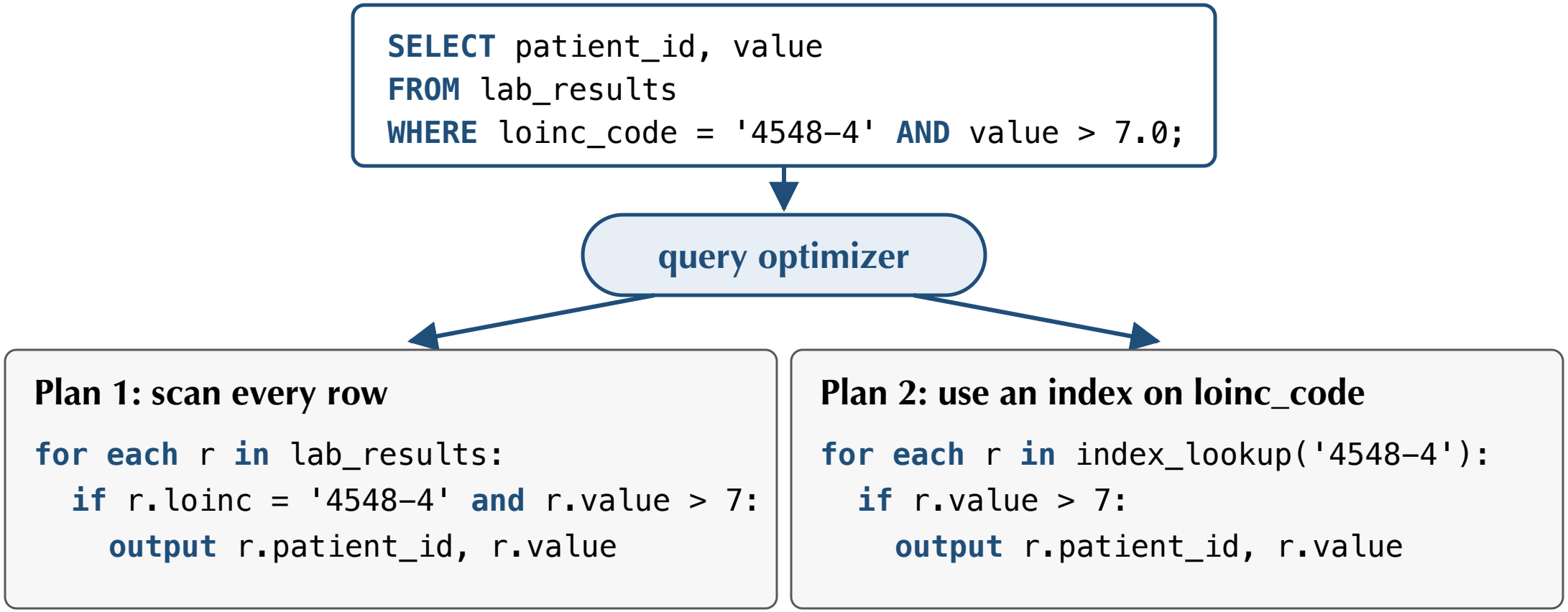
```sql
SELECT patient_id, value
FROM lab_results
WHERE loinc_code = '4548-4' AND value > 7.0;
```

Plan 1: scan every row

```
for each r in lab_results:
  if r.loinc = '4548-4' and r.value > 7:
    output r.patient_id, r.value
```

The same request in SQL states **what** result is wanted.

Declarative Queries



A single SQL query can be executed in many ways; choosing one is the job of the **query optimizer**.

Rows, Columns, and Row Grain

`lab_results`

lab_id	patient_id	loinc_code	value_numeric	unit	collected_at
L044	P001	4548-4	8.2	%	2026-03-15
L051	P001	4548-4	7.6	%	2026-06-20
L072	P002	4548-4	6.4	%	2026-04-02

- A **table** (relation) holds rows of one kind; a **column** (attribute) has one type
- The **row grain** is what a row represents. e.g., one lab result per row
- The **primary key** `lab_id` identifies each row

Schemas, Foreign Keys, Integrity Constraints

```
CREATE TABLE patients (
  patient_id TEXT PRIMARY KEY,
  mrn        TEXT UNIQUE NOT NULL
);
```

```
CREATE TABLE lab_results (
  lab_id        TEXT PRIMARY KEY,
  patient_id    TEXT NOT NULL REFERENCES patients(patient_id),
  loinc_code    TEXT NOT NULL,
  value_numeric REAL NOT NULL CHECK (value_numeric >= 0),
  unit          TEXT NOT NULL,
  collected_at  TEXT NOT NULL
);
```

Insert into `lab_results`	Result
`('L045', 'P999', '4548-4', 8.2, '%', '2026-03-16')`	
`('L044', 'P001', '4548-4', 7.9, '%', '2026-03-16')`	
`('L046', 'P001', '4548-4', -8.2, '%', '2026-03-16')`	
`('L047', 'P001', '4548-4', 8.2, NULL, '2026-03-16')`	
`('L048', 'P001', '4548-4', 82, '%', '2026-03-16')`	

Schemas, Foreign Keys, Integrity Constraints

```
CREATE TABLE patients (
  patient_id TEXT PRIMARY KEY,
  mrn        TEXT UNIQUE NOT NULL
);
```

```
CREATE TABLE lab_results (
  lab_id        TEXT PRIMARY KEY,
  patient_id    TEXT NOT NULL REFERENCES patients(patient_id),
  loinc_code    TEXT NOT NULL,
  value_numeric REAL NOT NULL CHECK (value_numeric >= 0),
  unit          TEXT NOT NULL,
  collected_at  TEXT NOT NULL
);
```

	Insert into `lab_results`	Result
✗	`('L045', 'P999', '4548-4', 8.2, '%', '2026-03-16')`	rejected: `FOREIGN KEY`
✗	`('L044', 'P001', '4548-4', 7.9, '%', '2026-03-16')`	rejected: `UNIQUE` (`lab_id`)
✗	`('L046', 'P001', '4548-4', -8.2, '%', '2026-03-16')`	rejected: `CHECK (value_numeric >= 0)`
✗	`('L047', 'P001', '4548-4', 8.2, NULL, '2026-03-16')`	rejected: `NOT NULL` (`unit`)
✓	`('L048', 'P001', '4548-4', 82, '%', '2026-03-16')`	accepted

Transactions and ACID

A **transaction** is a sequence of operations that runs all or nothing; concurrent transactions behave as if they never interleaved.

- **Atomicity:** all operations happen, or none do
- **Consistency:** constraints hold before & after
- **Isolation:** concurrent xacts run as if one at a time
- **Durability:** committed changes survive a crash

```
BEGIN;
INSERT INTO lab_results VALUES
('L044', 'P001', '4548-4', 8.2, '%', '2026-03-15');

-- ... 99 more INSERT statements ...

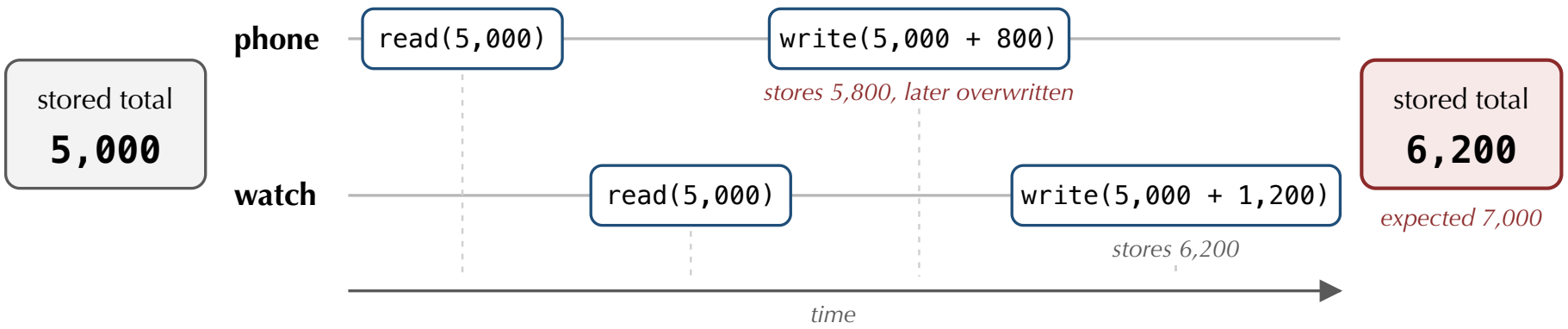
COMMIT;
```

If import crashes before `COMMIT`, transaction rolls back and no rows exist; rerunning stores 100 rows.

ACID: Examples

Property	Example	Without it
Atomicity	100 imported results commit together	40 of 100 rows after a crash
Consistency	result for unknown patient P999 rejected	a result with no patient
Isolation	phone and watch update one daily total at once	a lost update (below)
Durability	power fails just after `COMMIT`	a committed import lost

Lost update, the failure isolation prevents:



SQL Primer

```
SELECT collected_at, value_numeric          -- which columns
FROM lab_results                            -- which table
WHERE patient_id = 'P001'                   -- which rows
  AND loinc_code = '4548-4'
ORDER BY collected_at;                      -- in what order
```

```
collected_at | value_numeric
2026-03-15   | 8.2
2026-06-20   | 7.6
```

This is P001's HbA1c history: two stored results, oldest first. Each database system has its own SQL dialect, and dialects often disagree, especially on advanced features.

Learn more: SQLBolt, SQLZoo, and PostgreSQL Exercises (interactive exercises); Select Star SQL (interactive book); PostgreSQL tutorial

Joins

Which MRNs have an HbA1c above 7.0? What will the query return?

`patients`

patient_id	mrn
P001	MRN-0042
P002	MRN-0057

`lab_results`

lab_id	patient_id	loinc_code	value_numeric	collected_at
L044	P001	4548-4	8.2	2026-03-15
L051	P001	4548-4	7.6	2026-06-20
L072	P002	4548-4	6.4	2026-04-02

```
SELECT p.mrn, lr.collected_at, lr.value_numeric
FROM patients AS p
JOIN lab_results AS lr
  ON p.patient_id = lr.patient_id
WHERE lr.loinc_code = '4548-4'
  AND lr.value_numeric > 7.0;
```

`p`, `lr` are **aliases**; `ON` states which rows belong together.

Joins

Which MRNs have an HbA1c above 7.0? What will the query return?

`patients`

patient_id	mrn
P001	MRN-0042
P002	MRN-0057

`lab_results`

lab_id	patient_id	loinc_code	value_numeric	collected_at
L044	P001	4548-4	8.2	2026-03-15
L051	P001	4548-4	7.6	2026-06-20
L072	P002	4548-4	6.4	2026-04-02

```
SELECT p.mrn, lr.collected_at, lr.value_numeric
FROM patients AS p
JOIN lab_results AS lr
  ON p.patient_id = lr.patient_id
WHERE lr.loinc_code = '4548-4'
  AND lr.value_numeric > 7.0;
```

mrn	collected_at	value_numeric
MRN-0042	2026-03-15	8.2
MRN-0042	2026-06-20	7.6

`p`, `lr` are **aliases**; `ON` states which rows belong together.

Aggregation and GROUP BY

```
SELECT patient_id,
       COUNT(*)           AS n_results,
       AVG(value_numeric) AS mean_a1c
FROM lab_results
WHERE loinc_code = '4548-4'
GROUP BY patient_id;
```

patient_id	n_results	mean_a1c
P001	2	7.9
P002	1	6.4

- `GROUP BY` makes one output row per group, here per patient (defines the "grain")
- An **aggregate** summarizes each group's rows: `COUNT(*)`, `COUNT(DISTINCT …)`, `SUM`, `AVG`, `MIN`, `MAX`
- Evaluation order: `JOIN` produces combined rows → `WHERE` filters them → `GROUP BY` groups them into buckets and computes each aggregate over each bucket
- `DISTINCT` removes duplicates, as in `COUNT(DISTINCT patient_id)`

Many-to-Many Joins and Fan-Out

`conditions`

cond_id	patient_id	icd10
C1	P001	E11.9
C2	P001	I10
C3	P002	E11.9

`lab_results`

lab_id	patient_id	value
L1	P001	8.2
L2	P001	7.6
L3	P002	6.4

joined rows

cond_id	lab_id	value
C1	L1	8.2
C1	L2	7.6
C2	L1	8.2
C2	L2	7.6
C3	L3	6.4

```
SELECT c.cond_id, l.lab_id, l.value
FROM conditions AS c
JOIN lab_results AS l
  ON c.patient_id = l.patient_id;
```

- `COUNT(*)` = 5, but there are only 3 lab results!!
- `COUNT(DISTINCT l.lab_id)` = 3
- `SUM(l.value)` = 38.0 instead of 22.2
- `AVG(l.value)` = 7.60 instead of 7.40

LLMs for Schema and Query Generation

- **Schema generation:** given a CSV sample, generate `CREATE TABLE` statements with types and keys
- **Text-to-SQL:** given a schema and a question, generate a query
- **Document queries:** MongoDB Compass generates filters and aggregation pipelines from natural language
- **Benchmarks:** BIRD execution accuracy (share of generated queries whose results match the reference query's): GPT-4 55%, best system (Oct 2026) 83%, humans 93%; Spider 2.0 (enterprise schemas): 21%

Generated schemas and queries have known failure modes, shown next.

BIRD — Li et al., NeurIPS (2023); leaderboard, Oct 2026 · Spider 2.0 — Lei et al., ICLR (2025)

Generated Schemas: Failure Modes

```
mrn,name,test,result,units
MRN-0042,J. Smith,HbA1c,8.2,%
MRN-0042,J. Smith,LDL,131,mg/dL
```

```
CREATE TABLE lab_results (
  mrn    TEXT PRIMARY KEY,  -- one row per patient: the LDL row is rejected
  name   TEXT,              -- patient facts repeated on every result
  test   TEXT,
  result INTEGER,           -- PostgreSQL rounds 8.2 to 8
  units  TEXT
);
```

Fix: a `patients` table, a `lab_results` table keyed by result with a foreign key, and `REAL` values.

Generated SQL: Failure Modes

“What is the average HbA1c among patients taking metformin?”

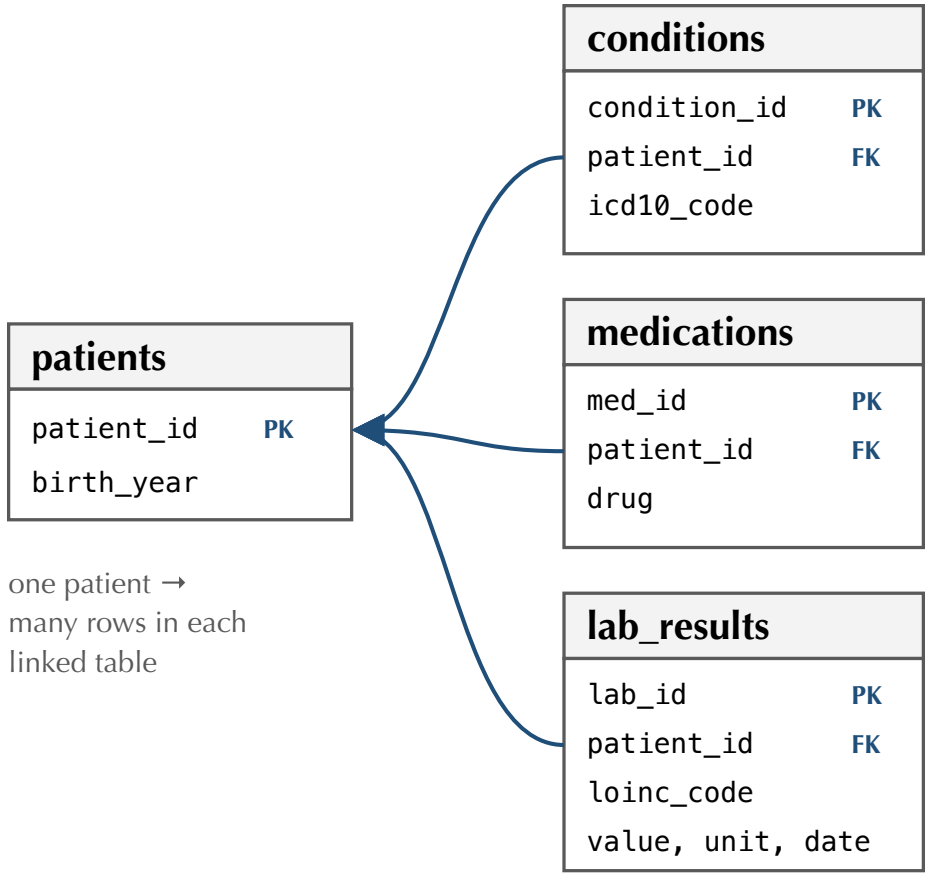
```
SELECT AVG(l.value)
FROM lab_results AS l
JOIN medications AS m ON l.patient_id = m.patient_id
WHERE m.drug = 'metformin' AND l.loinc_code = '4548-4';   -- a plausible average
```

- The same fan-out as before: a patient with two metformin orders counts each result twice; the exercise asks for the correct value
- Other common errors: counting rows instead of people, inventing a column, filtering on a label (`'A1C'`) instead of a code
- Run generated SQL read-only, then check it against a case you can compute by hand

Exercise: SQL on a Synthetic Health Database

Open the exercise page from the QR code. It runs DuckDB in your browser on a synthetic health database; nothing to install.

1. List P001's HbA1c results (`4548-4`), oldest first.
2. How many **people** have an HbA1c above 7.0?
3. How many HbA1c **results** are above 7.0?
4. Average HbA1c among people taking metformin: what are the potential risks in computing it?
5. Optional, with your OpenAI key: have a model draft the schema and the SQL, then check its work.



Break

8 minutes

Next: semantic operators in SQL.

Semantic Operators

A **semantic operator** is a relational operator—filter, map, join, aggregate, or top-k—whose condition is a natural-language instruction evaluated by a language model.

```
SELECT note_id, note_date
FROM notes
WHERE note_date >= DATE '2026-01-01'
  AND AI_FILTER(PROMPT('The patient has a penicillin allergy: {0}', note));
```

The DBMS evaluates the date predicate exactly; a model evaluates the semantic predicate for each row, at a per-token cost.

These features are very new, and their syntax and behavior differ across database systems.

Execution of a Semantic Operator

`WHERE ai_filter('The patient has a penicillin allergy: {0}', note)` is just a scalar expression!

Step	For note N3
Row	`N3`: “Hives after amoxicillin in 2019. Penicillin allergy documented.”
Fill the prompt template	“The patient has a penicillin allergy: Hives after amoxicillin in 2019. Penicillin allergy documented.”
Call model, batch w/ other rows	reply: “Yes”
Parse the reply into a SQL value	`TRUE` (an unparseable reply becomes `NULL` or an error)
Continue	N3 passes the `WHERE`; joins and aggregates use it

Common optimizations:

- Evaluate cheap predicates first
- Cache identical prompts
- **Model cascade:** a small model decides the confident rows; only the rest go to the large model

Semantic Operators and Relational Counterparts

Semantic operator	SQL counterpart	Health example
Filter	`WHERE`	notes that report a penicillin allergy
Map / extract	`SELECT` expression	HbA1c value, unit, and date from a report
Classify	`CASE`	assertion: patient, family, denied, hypothetical
Join	`JOIN … ON`	each note matched to the guideline it concerns
Aggregate	`GROUP BY`	one summary of a year of visit notes
Top-k	`ORDER BY … LIMIT`	the five notes most relevant to fatigue

Semantic Filter Syntax by System

Commercial database systems embed semantic operators in SQL syntax:

```
-- Snowflake Cortex
WHERE AI_FILTER(PROMPT('The patient has a penicillin allergy: {0}', note))

-- Google BigQuery
WHERE AI.IF(('The patient has a penicillin allergy: ', note))

-- MotherDuck (DuckDB)
WHERE prompt('Does the patient have a penicillin allergy? ' || note,  return_type := 'BOOLEAN')

-- DuckDB jev extension (decision model)
WHERE jev(note, 'the patient has a penicillin allergy')
```

Semantic Operators in Research Systems

Research systems are usually Python libraries over data frames or document collections:

```
# LOTUS: semantic operators on pandas data frames
notes.sem_filter("{note} says the patient has a penicillin allergy")

# DocETL: document-processing pipelines
frame = frame.filter(
    prompt="Does this note say the patient has a penicillin allergy? {{ input.note }}",
    output={"schema": {"is_allergic": "bool"}},
)
```

- LOTUS, DocETL, and Palimpzest also optimize the plan: model cascades, operator reordering, and prompt rewriting
- Several of their ideas, such as cascades, now appear in the commercial systems

LOTUS — Patel et al., PVLDB (2025); DocETL — Shankar et al., PVLDB (2025)

Extraction Function Syntax by System

```
-- Google BigQuery
SELECT AI.GENERATE(note, output_schema => 'value FLOAT64, unit STRING, test_date STRING')
FROM notes;

-- Snowflake Cortex
SELECT AI_EXTRACT(text => note,
                  responseFormat => {'value': 'HbA1c value?', 'unit': 'Unit?'})
FROM notes;

-- Databricks
SELECT ai_extract(note, '["hba1c_value", "unit", "test_date"]',
                  options => map('version', '2.1'))
FROM notes;

-- MotherDuck (DuckDB)
SELECT prompt(note, struct := {value: 'DOUBLE', unit: 'VARCHAR'}) FROM notes;
```

Exercise: Semantic SQL

7 minutes · exercise page, section 5

With your OpenAI key, `ai_filter` and `ai_extract` store the model's answers as tables that you join in SQL.

1. Keyword `ILIKE` versus `ai_filter`: who *currently* has a penicillin allergy?
2. Count the model calls; put the cheap predicate first
3. Turn on "reason first": does the answer change, and what does it cost?
4. Extract HbA1c, convert units in SQL, and check the evidence span
5. Read note N7, which contains an instruction to the model; then reword the question

Decision Models: Jev in DuckDB

A **decision model** answers a typed question about an input with a probability or a label, not generated text.

```
SELECT note_id, jev_prob(note, 'the patient currently has a penicillin allergy') AS p
FROM notes
WHERE jev(note, 'the note discusses a drug allergy');
```

- **Cost:** input only, $0.042 per million tokens; output is free
- **No generated text:** a probability or one of the options, so no sentence to hallucinate
- **Calibrated probabilities:** a probability of 0.8 should be right about 80% of the time
- These are design goals, not guarantees: check them on a labeled sample

Jev Functions

Function	Returns	Use
`jev(x, condition)`	`BOOLEAN`: probability ≥ `jev_threshold`	filter
`jev_prob(x, condition)`	probability	rank, review
`jev_choice(x, question, options)`	one of the options	classify
`jev_score(x, question, levels)`	probability-weighted level	rate

`x` is one column or a whole row.

Setting Up Jev in DuckDB

```
INSTALL jev FROM community;   -- DuckDB 1.5.5 or 1.5.6
LOAD jev;

SET jev_api_key = '...';                -- or the TYPESAFE_API_KEY environment variable
SET jev_model = 'jev-1.13.0';           -- pin a version (default: jev-latest)
SET jev_max_rows_per_statement = 1000;   -- refuse larger statements (default 0: no limit)

SELECT jev_stats();   -- requests, rows evaluated, cache hits, estimated cost
```

- Every row that reaches a `jev` function is sent to TypeSafe's API: use synthetic or approved data
- `jev_api_url` points the extension at any compatible server, such as a local model
- To try Jev without installing anything, paste a Jev API key at the top of the exercise page (section 6)

Jev: Filtering by an Attribute

```
SELECT note_id, note_date, note
FROM notes
WHERE note_date >= DATE '2026-04-01'
  AND jev(note, 'the patient currently has a penicillin allergy');
```

- `jev()` is true if probability > `jev_threshold` (default 0.5); `jev(note, '…', 0.9)` sets threshold.
- Non-AI predicates run first to reduce costs.
- `jev_prob(note, '…')` with the same condition uses cached answers.

Jev: Adding a Semantic Column

```
CREATE TABLE notes_labeled AS
SELECT *,
       jev_choice(note, 'whose penicillin allergy is described?',
                  ['patient', 'family member', 'denied', 'hypothetical', 'none'])
         AS allergy_status,
       jev_prob(note, 'the patient currently has a penicillin allergy') AS p_allergy
FROM notes;

SELECT allergy_status, count(*) FROM notes_labeled GROUP BY allergy_status;
```

- The model runs once, at `CREATE TABLE`; later queries read the stored columns at no cost
- Stored labels can be reviewed, corrected, and given provenance: model version and date

Normalizing the Input Is Caching

`prescriptions`

order_id	drug
O1	metformin
O2	metformin
O3	metformin
O4	insulin glargine

`drugs`

drug_id	drug	class
D1	metformin	metformin
D2	insulin glargine	insulin

`orders`

order_id	drug_id
O1	D1
O2	D1
O3	D1
O4	D2

- `jev_choice(drug, …)` over `prescriptions`: **4** rows sent, three of them the same value
- Over the normalized `drugs` table: **2** rows sent; `orders` get `class` by a join

Normalizing stores each value once, so the model runs once per value. If the `jev` call did not cache, normalizing would do the caching.

Exercise: Jev

4 minutes · exercise page, section 6

Paste a Jev API key at the top of the page; without one, a mock shows the output format.

1. **Filter:** run `jev_prob` on the notes; set the threshold and read the rows sent
2. **Classify:** run `jev_choice`; check notes N4, N5, and N6 by hand
3. **Normalize:** classify every medication order, then only the distinct drug names; compare the rows sent

More examples: 5 Jev use cases for analytics, with real queries and datasets (MotherDuck)

Risks of Semantic Operators

Cost multiplies with joins. 1,000,000 notes; patients average 4 medications:

```
SELECT n.note_id, m.drug
FROM notes n
JOIN medications m USING (patient_id)
WHERE ai_filter('… penicillin allergy …', n.note);
```

- The model runs once per **joined** row: 4,000,000 calls, about $2,400, instead of 1,000,000 calls, about $600
- What can help: filter the notes first, then join
- A semantic join of 10,000 notes × 500 guideline passages is 5,000,000 calls

Additional Risks/Failure Modes

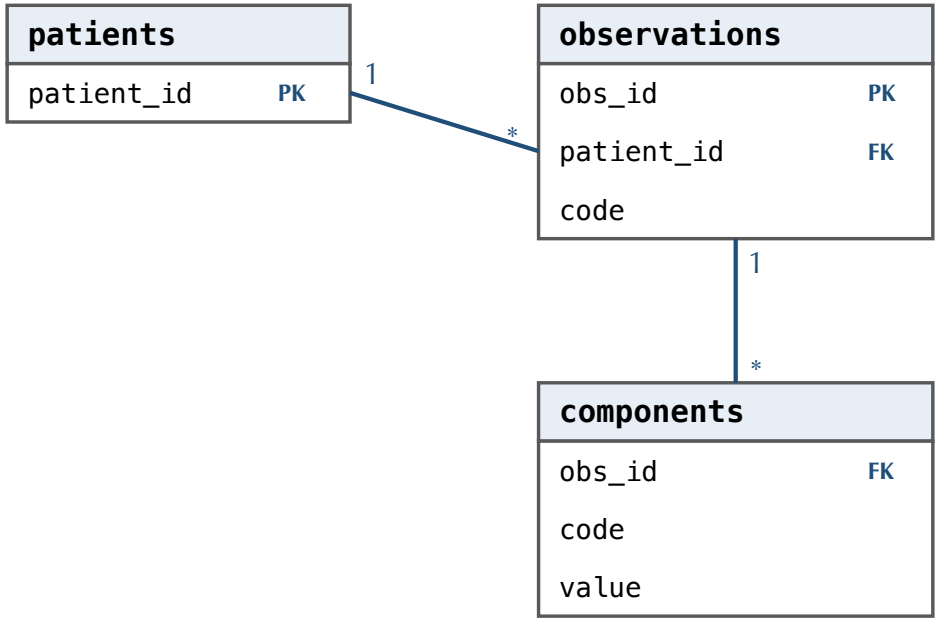
- **Prompt sensitivity:** rewording the demo question changed five matches to four
- **Unsupported values:** an answer with no supporting span in the note
- **Prompt injection:** note N7 instructs the model to answer yes, and the model complied
- **Privacy:** every row is sent to the model provider; health data needs an approved deployment or a local model
- **Model drift:** a provider's model update can change results for the same query

Document Stores

The document model, querying, and schemas.

Documents and Collections

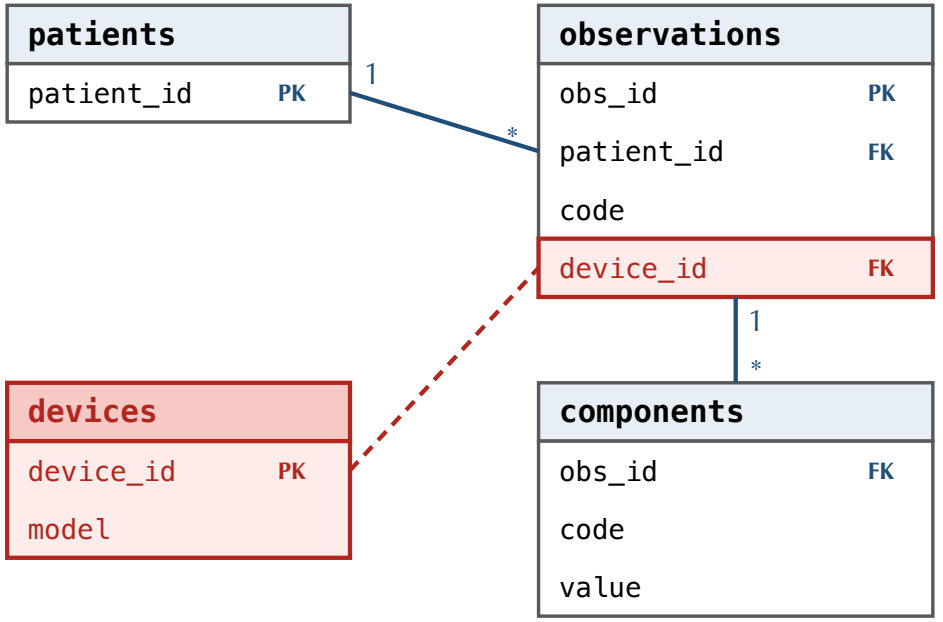
```
{
  "_id": "obs-bp-07",
  "subject": {"reference": "Patient/P001"},
  "code": "85354-9",
  "component": [
    {"code": "8480-6", "value": 138},
    {"code": "8462-4", "value": 86}
  ]
}
```



A **document** is a self-contained JSON object with an identifier; a **collection** groups documents that need not share one structure. This document **embeds** the components and **references** the patient; the relational schema needs a child table for components and foreign keys.

Documents and Collections

```
{
  "_id": "obs-bp-07",
  "subject": {"reference": "Patient/P001"},
  "code": "85354-9",
  "component": [
    {"code": "8480-6", "value": 138},
    {"code": "8462-4", "value": 86}
  ],
  "device": {"model": "Omron BP7450"}
}
```



One new field in the documents needs a new table and a new column in the relational schema. The schema is often not known in advance:

- Different data providers send different fields
- Requirements change as the application grows
- Early in development, the right structure is still unclear

Schema-on-Write and Schema-on-Read

Schema-on-write checks structure when data are stored. **Schema-on-read** stores data first and imposes structure when a reader queries it.

	Schema-on-write	**Schema-on-read**
The reading gains a `device` field (previous slide)	add a `devices` table and column first	store it now
A value arrives as text, `"138"`	rejected at insert	found by each reader
Who absorbs variation	the writer, once	every reader, every time

Querying Documents in MongoDB

```
// collection obs
{ _id: "obs-1", subject: "P001",
  code: "4548-4", value: 8.2 }
{ _id: "bp-7", subject: "P001", code: "85354-9",
  component: [ { code: "8480-6", value: 138 },
               { code: "8462-4", value: 86 } ] }
{ _id: "bp-8", subject: "P002", code: "85354-9",
  component: [ { code: "8480-6", value: 128 },
               { code: "8462-4", value: 94 } ] }
```

```
db.obs.find({ code: "4548-4",
              value: { $gt: 7.0 } })
// → obs-1

db.obs.find({ "component.code": "8462-4" })
// → bp-7, bp-8
```

- The query is a document shaped like the data: keys name fields, values say what to match
- A key starting with `$` is an operator, not a field: as a value, `{ $gt: 7.0 }` compares that field; at the top level, `$or: [ … ]` combines conditions
- Dot notation reaches inside: `"component.code"` matches if **any** component has that code

MongoDB Queries: Array Semantics

```
// collection obs
{ _id: "obs-1", subject: "P001",
  code: "4548-4", value: 8.2 }
{ _id: "bp-7", subject: "P001", code: "85354-9",
  component: [ { code: "8480-6", value: 138 },
               { code: "8462-4", value: 86 } ] }
{ _id: "bp-8", subject: "P002", code: "85354-9",
  component: [ { code: "8480-6", value: 128 },
               { code: "8462-4", value: 94 } ] }
```

“Find readings with diastolic pressure above 90.”

```
db.obs.find({ "component.code": "8462-4",
              "component.value": { $gt: 90 } })
// → bp-7, bp-8: in bp-7 the code
//   matches diastolic, the value systolic

db.obs.find({ component: { $elemMatch: {
  code: "8462-4", value: { $gt: 90 } } } })
// → bp-8: both hold for one component
```

Without `$elemMatch`, each condition may be satisfied by a different array element.

JSON Schemas

```
db.createCollection("obs", { validator: {
  $jsonSchema: {
    bsonType: "object",
    required: ["subject", "code"],
    properties: {
      subject: { bsonType: "string" },
      code:    { bsonType: "string" },
      value:   { bsonType: "number",
                 minimum: 0 },
      component: { bsonType: "array",
        items: { bsonType: "object",
                 required: ["code", "value"] } }
} } } })
```

JSON Schema	Relational counterpart
`bsonType`	column type
`required`	`NOT NULL`
`minimum`, `enum`	`CHECK`
nested `properties`, `items`	child table
none	foreign key

A validator checks each document at insert.
References such as `subject` are not checked.

Schema Inference in DuckDB

FHIR Bulk Data exports newline-delimited JSON (NDJSON): one resource per line.

`Observation.ndjson`, one line shown wrapped:

```
{"subject": {"reference": "Patient/P001"},
 "code": {"coding": [{"code": "4548-4"}]},
 "valueQuantity": {"value": 8.2, "unit": "%"},
 "effectiveDateTime": "2026-03-15T09:10:00-04:00"}
```

Loaded with `read_json`, the inferred columns:

column	inferred type
`subject`	`STRUCT(reference VARCHAR)`
`code`	`STRUCT(coding STRUCT(code VARCHAR)[])`
`valueQuantity`	`STRUCT(value DOUBLE, unit VARCHAR)`
`effectiveDateTime`	`VARCHAR`

Then query it:

```
SELECT subject.reference   AS patient,
       valueQuantity.value AS value,
       effectiveDateTime::TIMESTAMPTZ AS taken_at
FROM read_json('Observation.ndjson')
WHERE code.coding[1].code = '4548-4';
```

```
patient      | value | taken_at
Patient/P001 | 8.2   | 2026-03-15 13:10:00+00
```

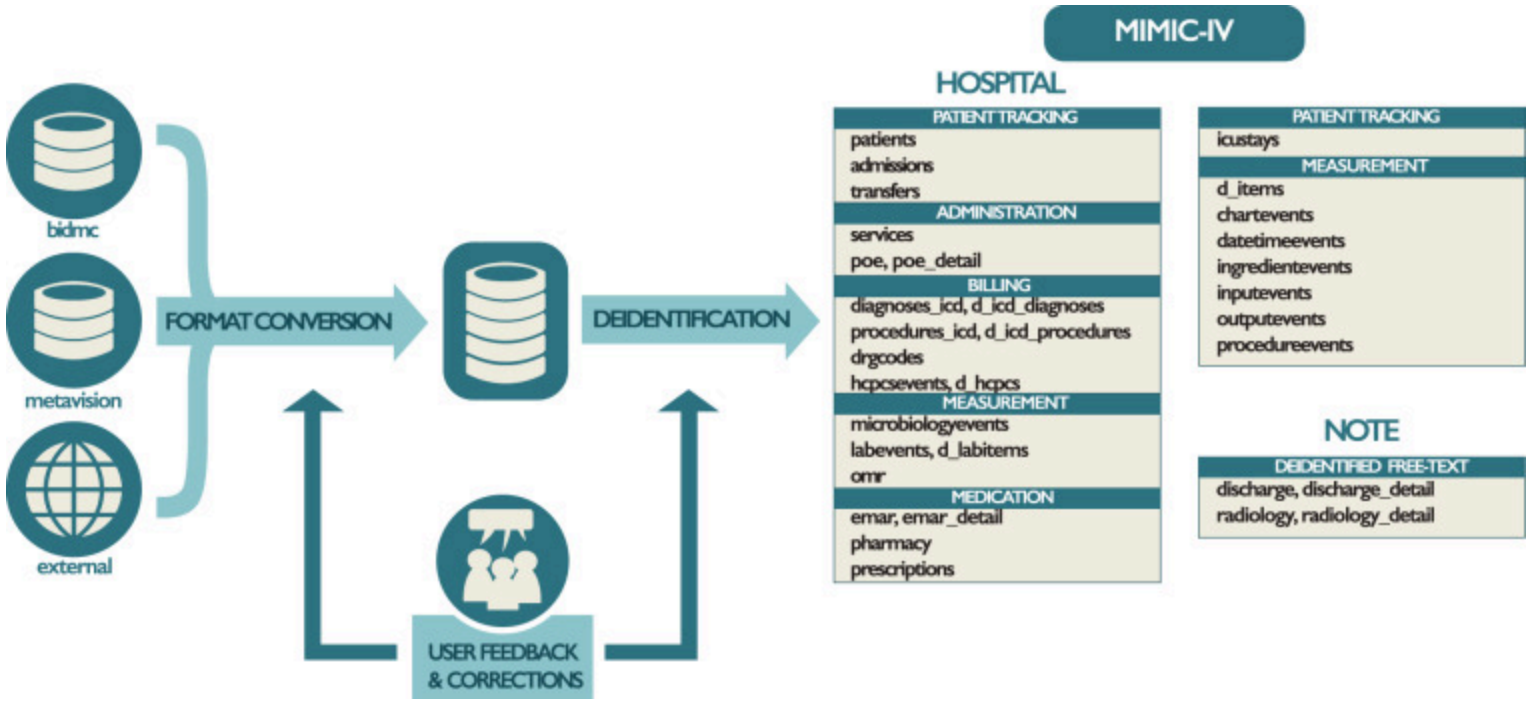
- Nested JSON becomes `STRUCT` and list columns, reached with dot notation
- The date was inferred as text, so it needs a cast; inference decides types when the data are read

Information Extraction, Integration, and Cleaning

From text and mixed sources to clean, traceable records.

Data Curation Pipeline

Ingest → Extract → Normalize → Integrate → Load



MIMIC-IV data pipeline — Johnson et al., Scientific Data (2023)

Data Curation Pipeline

Ingest → Extract → Normalize → Integrate → Load

- **Ingest:** bring data from each source into the system, with its source and identifier
- **Extract:** derive structured information from text and documents
- **Normalize:** make values consistent within a dataset: units, codes, dates
- **Integrate:** reconcile across datasets and sources: the same person, the same test, duplicates, conflicts
- **Load:** write the cleaned rows, with provenance, into the trusted database

Step 1: Ingest

Two reports arrive as text files:

```
report_A.txt   HEMOGLOBIN A1C ...... 8.2 %  H  (4.0–5.6)   Collected: 03/15/2026
report_B.txt   HbA1c 58 mmol/mol (IFCC). Sample taken 05/06/2026.
```

```
CREATE TABLE documents AS
SELECT filename AS doc_id, content AS text, now() AS ingested_at
FROM read_text('reports/*.txt');
```

- Each document is stored unchanged, with its source and the time it arrived
- Later steps can be rerun from the raw text when an extractor or a rule improves

Step 2: Extract

Report A, as ingested:

```
HEMOGLOBIN A1C ...... 8.2 %  H  (4.0–5.6)   Collected: 03/15/2026
```

```
CREATE TABLE extracted AS
SELECT doc_id,
       ai_extract('Return test, value, unit, date, and the exact evidence text', text) AS r
FROM documents;
```

```
{"test": "HEMOGLOBIN A1C", "value": 8.2, "unit": "%",
 "date": "03/15/2026", "evidence": "HEMOGLOBIN A1C ...... 8.2 %"}
```

Extraction copies what the text says. It does not yet choose a code, convert a unit, or decide the date order.

Step 2: Extraction Methods

Method	On Report A	Typical failure
Rules and dictionaries	regex `A1C\D*([\d.]+) *%` → `8.2`	misses "A1c of 8.2"
Clinical NLP pipeline (cTAKES)	span "HEMOGLOBIN A1C" → a lab-test concept	shifts across institutions
LLM with an output schema	text → `{"value": 8.2, "unit": "%"}`	can invent a value
Decision model (Jev)	`jev_choice(text, 'which test?', ['HbA1c', 'LDL'])` → `HbA1c`	no numbers, no explanation

GPT-4 clinical entity recognition: F1 0.86 with prompt engineering, against 0.90 for a fine-tuned BioClinicalBERT.

Step 3: Clean and Normalize

One source, the clinic that sent Report B, cleaned with its own conventions:

`clinic_b`, as extracted

doc_id	test	value	unit	date
B1	HbA1c	58	mmol/mol	05/06/2026
B2	HbA1c	64	mmol/mol	12/01/2026
B3	Glucose	7.2	mmol/L	12/01/2026

normalized

doc_id	loinc_code	value	unit	collected_at
B1	4548-4	7.5	%	2026-06-05
B2	4548-4	8.0	%	2026-01-12
B3	2345-7	130	mg/dL	2026-01-12

```
SELECT b.doc_id, m.loinc_code, m.std_unit AS unit,      -- test_map: label → LOINC, unit
       CASE b.unit WHEN 'mmol/mol' THEN round(0.09148 * b.value + 2.152, 1)
                   WHEN 'mmol/L'   THEN round(b.value * 18.016)
                   ELSE b.value END AS value,
       strptime(b.date, '%d/%m/%Y') AS collected_at     -- this source writes day first
FROM clinic_b AS b JOIN test_map AS m ON m.label = b.test;
```

Step 3: Common Cleaning Rules

Need	Example	Typical rule
Units	HbA1c in % and mmol/mol; height in inches and cm	convert to one unit; keep the original
Missing units	15.8% of heights in All of Us had no unit	infer from plausible ranges; flag the row
Dates	`05/06/2026`, day first or month first	parse with the source's convention; store ISO 8601
Codes/Labels	HbA1c, HEMOGLOBIN A1C, A1c	map to one code, LOINC 4548-4
Outliers	an HbA1c of 82%	range check
"Nullish" values	7 or 9 for "refused" or "don't know" in surveys	convert to `NULL` with a rationale

Step 4: Integrate

Two records from different sources, each fine on its own:

source	name	birth_date	loinc_code	value_pct	collected_at
portal	John Smith	1968-02-03	4548-4	8.2	2026-03-15
report_A	J. Smith	NULL	4548-4	8.2	2026-03-15

Same person? Same test? Integration compares records **across** sources, pair by pair:

```
SELECT p.*, r.*
FROM portal AS p JOIN reports AS r
  ON p.loinc_code = r.loinc_code AND p.collected_at = r.collected_at   -- blocking
WHERE jaro_winkler_similarity(p.name, r.name) > 0.8;                      -- similar names
```

- Comparing every pair costs n × m: 10,000 × 10,000 records is 100,000,000 comparisons
- **Blocking** compares only plausible pairs, here the same test on the same date
- Then decide: the same person, one test recorded twice; keep one row and record why

Step 5: Load

The goal is a **trusted database** that applications can rely on, so only rows that pass the checks are loaded into it.

```
BEGIN;
INSERT INTO lab_results                  -- trusted table: keys, foreign keys, NOT NULL
SELECT * FROM integrated WHERE value_pct BETWEEN 3 AND 20;
INSERT INTO review_queue                 -- everything else waits for a person
SELECT * FROM integrated WHERE value_pct NOT BETWEEN 3 AND 20;
COMMIT;
```

- The data model and constraints check syntax; steps 2–4 handle meaning: units, codes, dates, and people
- Rows that fail checks are stored to be reviewed.
- Errors can still exist, so trusted table needs cleaning. Provenance helps debug pipeline.

Example Failure Modes in Curation

Step	Failure	Example
Extract	negation ignored	“No penicillin allergy” stored as an allergy
Extract	wrong experiencer	“Mother has breast cancer” stored as the patient’s
Extract	invented value	a value with no supporting span in the text
Normalize	unit misread	58 mmol/mol stored as 58%
Normalize	date order	`05/06/2026` read as May 6
Integrate	wrong person	Report B matched to P001 on the name alone
Integrate	duplicate	the March test counted twice

Provenance

No step is guaranteed to be correct. Provenance tracks where each fact came from and every decision that produced it.

Step	Example for row L-B1: HbA1c 7.5%, 2026-06-05, P001
Ingest	`report_B.txt`, uploaded by the user, 2026-06-12
Extract	span, characters 0–24: “HbA1c 58 mmol/mol (IFCC)”; model and prompt version
Normalize	59261-8 → 4548-4; 58 mmol/mol → 7.5% by `ifcc_to_ngsp` v1.0; date read day first
Integrate	matched to P001 by rule `name_dob` v2; not a duplicate of L-A1 (different date)
Load	passed the plausibility rule; confirmed by the user, 2026-06-12

Summary

1. A **data model** defines structures, relationships, constraints, and operations; a file format only lays out bytes.
2. **Schemas and constraints** catch obvious, syntactic errors when data are written; they cannot catch well-formed values that are wrong.
3. **Grain**, what one row represents, decides whether counts, sums, and averages are correct, especially after joins.
4. **Documents** suit data whose schema is not known in advance; the rules still have to be declared somewhere.
5. **Extraction and curation** deal with meaning: units, codes, dates, negation, and which person a record belongs to.
6. **Provenance** records how each fact was produced, so errors found later can be traced and fixed.

Exercises!
Thanks for the Attention
Q&A